Langage Interprété

Langages interprétés et écosystèmes modernes

1ère année Cycle ingénieur ISEN – Lille

2026/2027

Abderrahmane Kasmi

Séquence 1

Langages interprétés et écosystèmes modernes

01 **Compilation et interprétation**

02 **Écosystèmes logiciels modernes**

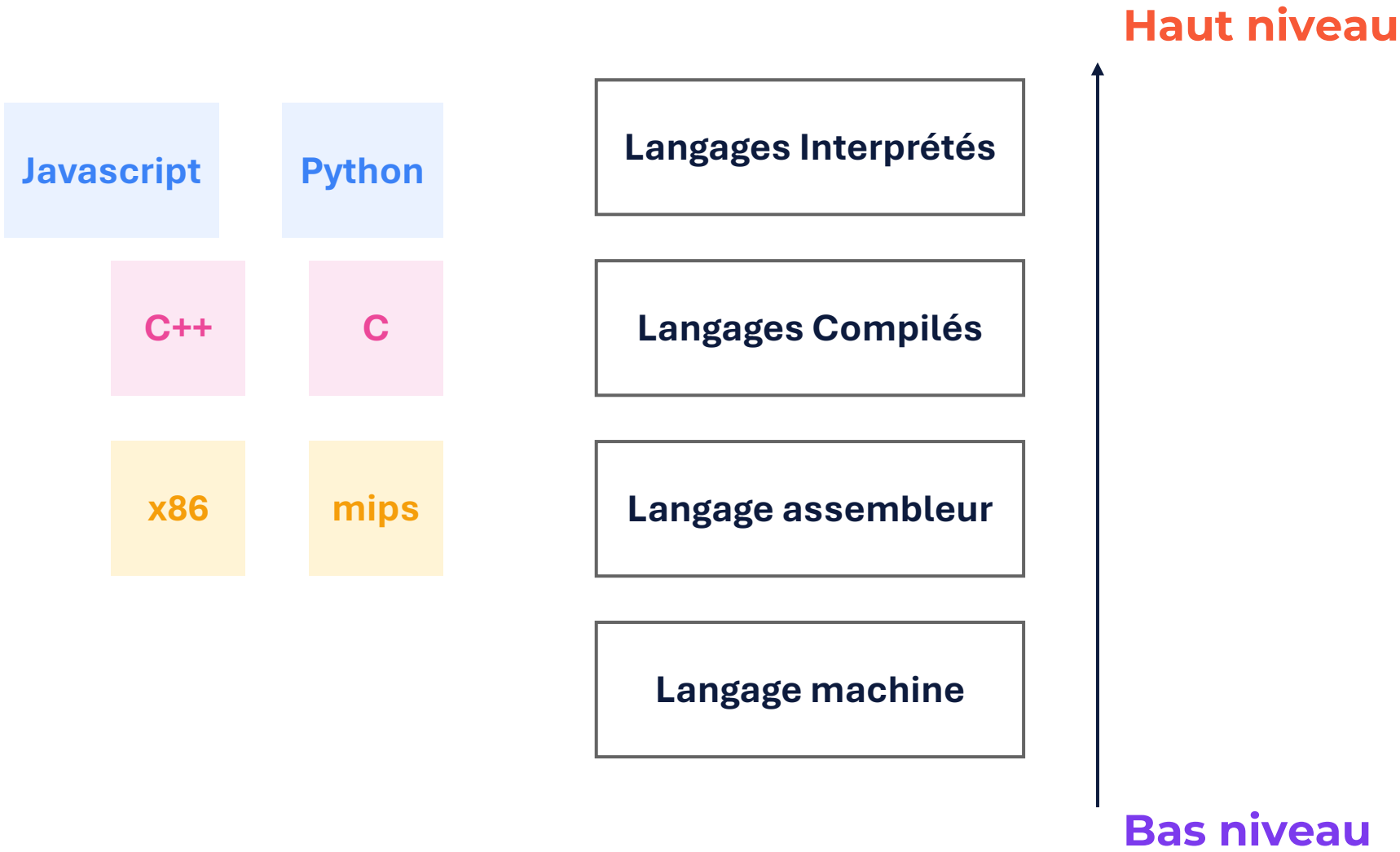
03 **Gestionnaire de paquets**

04 **Documentation et utilisation de bibliothèques**

01

Compilation et Interprétation

Abstraction et langages de programmation



Compilation

hello.c

```c
#include <stdio.h>

int main() {
    printf("Hello, World!\n");
    return 0;
}
```

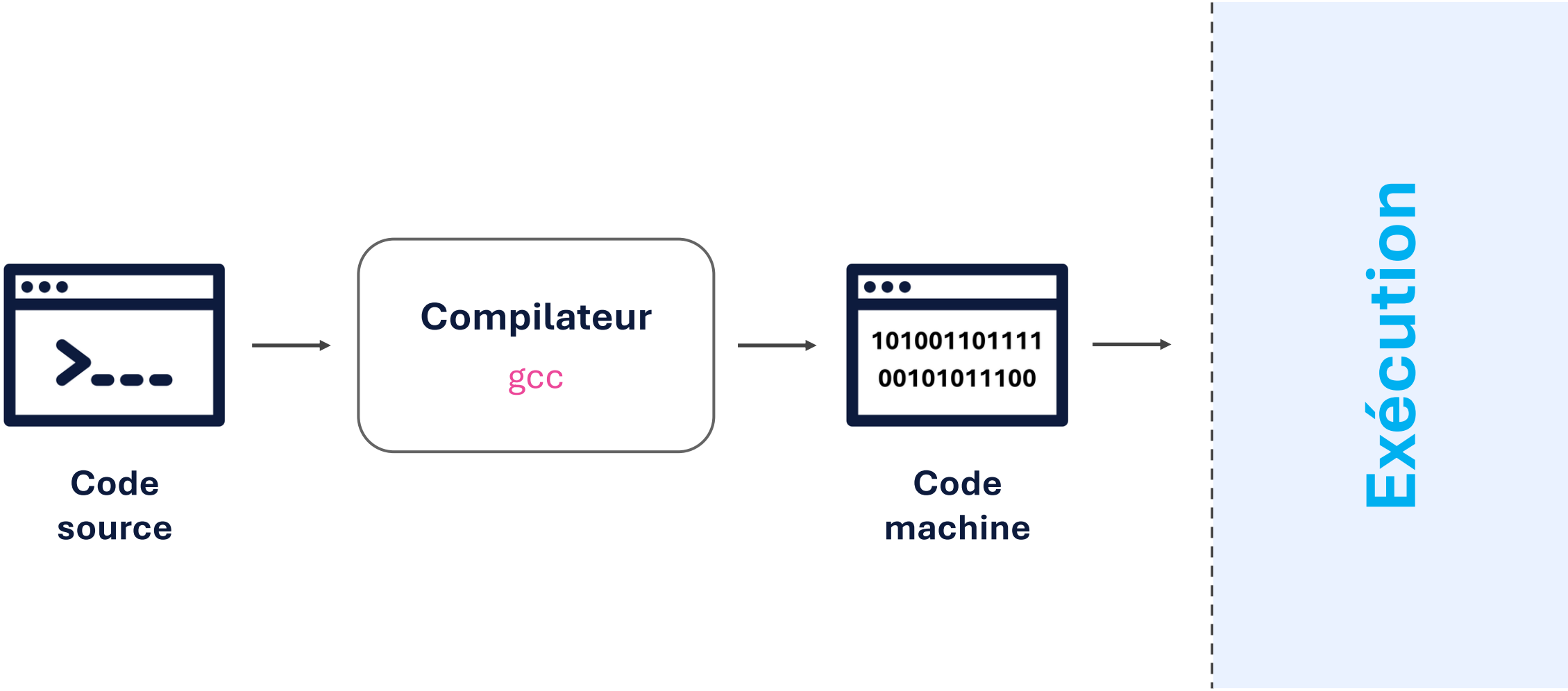
Compilation

```
$ gcc hello.c -o hello
```

Exécution

```
$ ./hello
```

Compilation



Compilation

- Traduction vers le langage machine
- Vérification de certaines erreurs (syntaxique, typage, ...)
- Optimisations adaptées à la machine cible

Interprétation

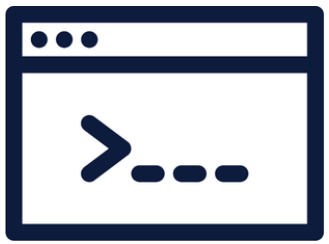
hello.py

```python
print("Hello, World!")
```

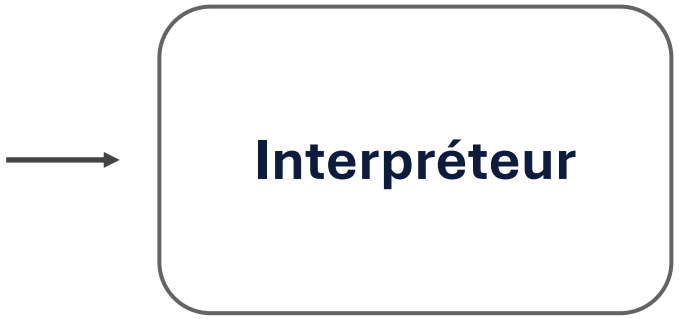
Exécution

```
$ python hello.py
```

Interprétation



Code source



Exécution

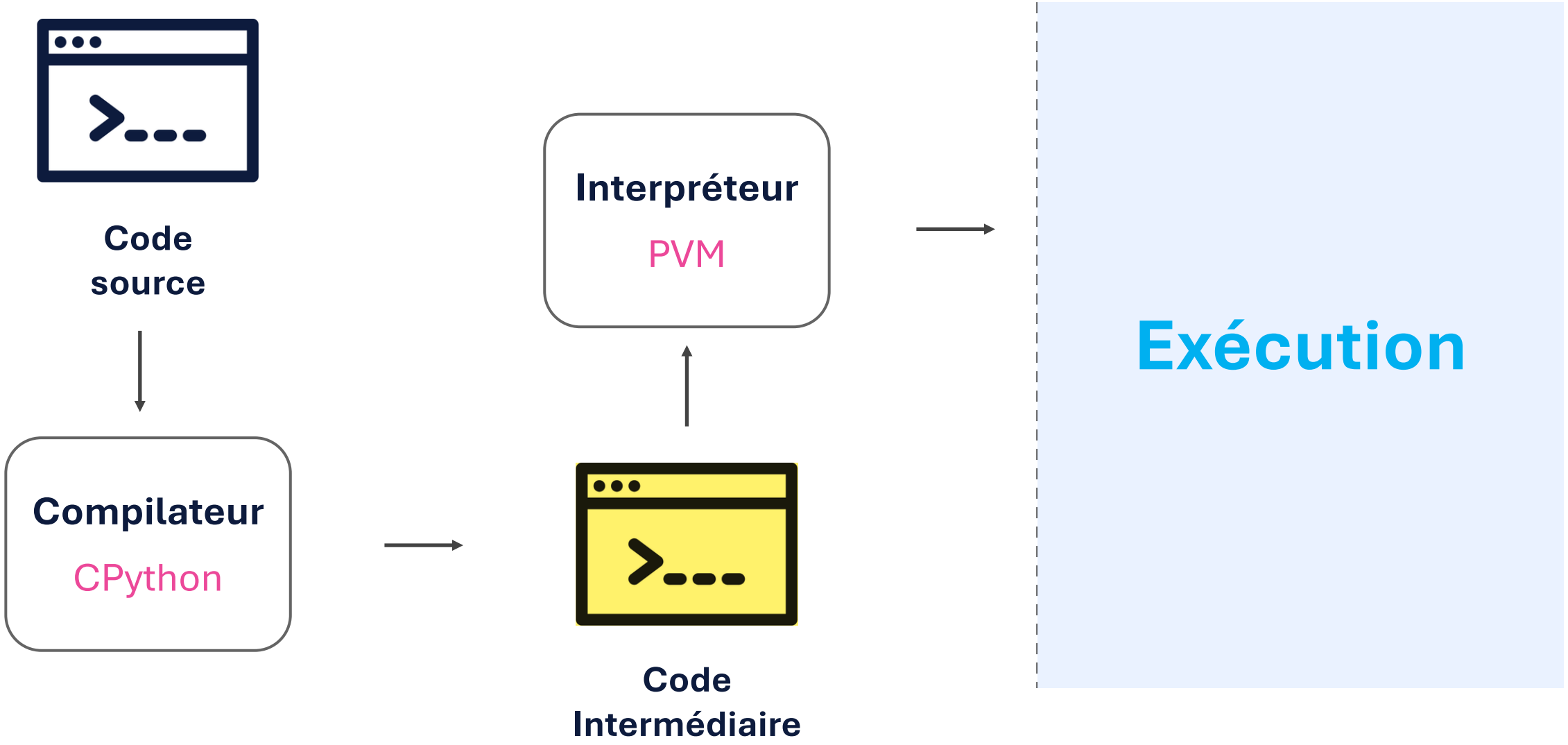
Interprétation

- Traduction vers le langage machine en temps réel
- Vérification des erreurs à l’exécution

Compilation vs Interprétation

	Compilation	Interprétation
Traduction	Tout le code est traduit en **langage machine (binaire)** en une seule fois, avant l'exécution	Le code est traduit et exécuté **ligne par ligne** au moment du lancement
Vitesse d'exécution	**Très rapide** (optimisée directement pour le processeur)	**Plus lente** (surcoût lié à la traduction en temps réel)
Détection des erreurs	**Au moment de la compilation** (erreurs de syntaxe et de typage bloquent la génération du binaire)	**À l'exécution** (le programme s'arrête lorsqu'il rencontre la ligne problématique)
Portabilité	**Faible** (le fichier binaire généré dépend du système d'exploitation et de l'architecture)	**Élevée** (le même code source s'exécute partout, à condition d'avoir l'interpréteur)
Cycle de développement	**Plus lent** (nécessite l'étape : modification → re-compilation → exécution)	**Plus rapide** (modifications testables immédiatement)

Python


Code
source
Compilateur
CPython
Code
Intermédiaire
Interpréteur
PVM
Exécution

Comparaison

	Code source	Code machine	Code intermédiaire
Cible	Humain (développeur)	Processeur	PVM
Lisibilité			?
Portabilité			
Prêt pour éxécution			

02

Écosystèmes logiciels modernes

Les Piliers de l’Ecosystème

Bibliothèques Tierces

Un code écrit par d'autres personnes et qui est réutilisable par tout le monde

Dépôts Centralisés

L’endroit ou les bibliothèques tierces sont stockés

Outils

Tous ce qui a été créé pour facilité le travail des développeurs

Ecosystème: les trois couches

Bibliothèques Tierces
matplotlib, pandas, numpy, requests

Bibliothèques Standard
math, os, sys, datetime, json

Code Développeur

03

Gestionnaire de paquets

Un gestionnaire de paquet set à:

- Installer des bibliothèques tierces
- Gérer les versions et mettre à jour les bibliothèques
- Désinstaller les bibliothèques

04

Documentation et utilisation de bibliothèques

Ou trouver la documentation

- Dépôt centralisé
- Site dédié de la bibliothèque
- Dépôt sur GitHub
- Dans le code source

Types de documentation

- Les fichiers README
- Documentation officielle (API reference)
- Guides / Tutoriels
- Docstrings